DATABASE ADMINISTRATION

Hackathon Project Documentation Template – Level 2

STUDENT NAME	:	LUXAN RESHMA. R
YEAR	:	2024-2027
COLLEGE CODE	:	BRU3J
COLLEGE NAME	:	TIRUPPUR KUMARAN COLLEGE FOR WOMEN
DEPARTMENT	:	BACHELOR OF COMPUTER APPLICATION
SEMESTER	:	5th SEMESTER
REGISTER NO	:	2422J0620
NMID	:	6790F77796EF6F8A6D7493E555F23526@VTPT2026.in
DATE	:	28/10/2026
GITHUB LINK	:	28/10/2026

1. Project Title

Student Examination Management System

The Student Examination Management System is a relational database application designed to manage student information, subjects, examinations, examination schedules, marks, grades, and academic results.

The system uses SQL to store, update, validate, and retrieve examination data. It generates student-wise results and subject-wise performance reports accurately and efficiently

2. Abstract

The Student Examination Management System is developed to simplify the management of student examination records. In educational institutions, maintaining examination schedules, entering marks, calculating grades, and preparing results manually can be time-consuming and may lead to errors.

This project uses a relational database to store student details, subject information, examination schedules, and marks in separate but related tables. Primary keys, foreign keys, unique constraints, and validation rules maintain data accuracy and consistency.

The system calculates total marks, percentages, grades, and pass/fail results using SQL queries. It also handles absent students, prevents duplicate mark entries, and generates academic performance reports.

The project demonstrates the practical use of database management concepts such as table creation, relationships, constraints, joins, aggregate functions, views, stored procedures, transactions, and indexes.

3. Objectives

The main objectives of the project are:

- To create and maintain student examination records.
- To create related tables for students, subjects, examinations, schedules, and marks.
- To establish primary keys and foreign keys to maintain relationships between tables.
- To validate obtained marks against the maximum marks allowed.
- To prevent duplicate mark entries for the same student, subject, and examination.
- To calculate totals, percentages, grades, and pass/fail results automatically.
- To manage absent students correctly.
- To generate student-wise and subject-wise performance reports.
- To calculate average, highest, and lowest marks using aggregate functions.
- To improve database accuracy, consistency, and retrieval performance.

4. Problem Statement

Educational institutions need to maintain a large amount of examination data, including student details, subjects, examination dates, marks, and results.

When this information is maintained manually or in separate files, several problems may occur. These include duplicate records, incorrect marks, calculation errors, missing examination details, and difficulties in preparing reports.

The proposed Student Examination Management System solves these problems by storing examination information in a centralized relational database.

The database validates marks, prevents duplicate entries, calculates grades and results, and generates reports using SQL queries. This reduces manual work and improves the reliability of examination records.

5. Tools and technology used

The following tools and technologies are used in this project.

Tool	Purpose
MySQL 8.0	Relational database management
MySQL Workbench	Creating tables and executing SQL queries
SQL	Creating, updating, retrieving, and managing data
ER Diagram	Representing database entities and relationships

SQL commands used: CREATE, INSERT, UPDATE, DELETE, SELECT, JOIN, GROUP BY, and aggregate functions.
Additional database features include constraints, views, stored procedures, triggers, transactions, and indexes.

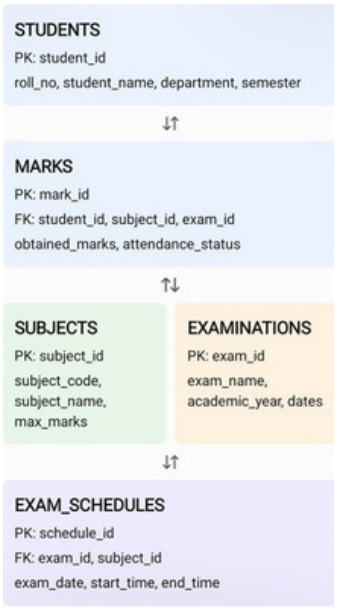
6. Database Design

Database design is the process of organizing information into related tables. The proposed system contains five main tables:

1. Students
2. Subjects
3. Examinations
4. Exam Schedules
5. Marks

Each table stores specific information. Relationships between tables are established using primary keys and foreign keys.
This design reduces unnecessary duplication and maintains data integrity.

6.1 Entity Relationship (ER) Diagram



Explanation: A student can have multiple mark records. A subject can be included in multiple examinations, and an examination can contain multiple scheduled subjects. Each mark record is linked to a valid student, subject, and examination schedule.
PK = Primary Key; FK = Foreign Key.

6.2 Database Structure

Table 1: Students
This table stores basic information about students.

Column	Data type	Description
student_id	INT	Primary key
roll_no	VARCHAR(20)	Unique roll number
student_name	VARCHAR(100)	Student name
department	VARCHAR(60)	Department na
semester	TINYINT	Semester num

Table 2: Subjects
This table stores the details of subjects offered by the institution.

Column	Data type	Description
subject_id	INT	Primary key
subject_code	VARCHAR(20)	Unique subject code
subject_name	VARCHAR(100)	Name of subject
max_marks	DECIMAL(5,2)	Maximum marks

Table 3: Examinations
This table stores examination information.

Column	Data type	Description
exam_id	INT	Primary key
exam_name	VARCHAR(100)	Examination name
academic_year	VARCHAR(9)	Academic year
start_date	DATE	Examination st date
end_date	DATE	Examination en date

Table 4: Exam Schedules
This table stores the date and time of each subject examination.

Column	Data type	Description
schedule_id	INT	Primary key
exam_id	INT	Foreign key
subject_id	INT	Foreign key
exam_date	DATE	Examination date
start_time	TIME	Start time
end_time	TIME	End time

Table 5: Marks
This table stores the marks obtained by students.

Column	Data type	Description
mark_id	INT	Primary key
student_id	INT	Foreign key
subject_id	INT	Foreign key
exam_id	INT	Foreign key
obtained_marks	DECIMAL(5,2)	Marks obtained
attendance_status	ENUM	PRESENT or ABSENT
updated_at	TIMESTAMP	Last update time

A unique constraint on student_id, subject_id, and exam_id prevents duplicate mark records.

7. SQL Operations

SQL is used to create tables, insert data, update records, retrieve information, and generate reports

7.1 Constraints and Relationships

Constraints maintain the correctness of database records.

- PRIMARY KEY: Uniquely identifies each record.
- FOREIGN KEY: Connects related tables.
- UNIQUE: Prevents duplicate values.
- NOT NULL: Ensures required information is provided.
- CHECK: Validates conditions such as semester range and examination dates.
- TRIGGERS: Validate marks against the maximum marks for a subject.

EXAMPLE:

```
CREATE TABLE students (
 student_id INT PRIMARY KEY AUTO_INCREMENT,
   roll_no VARCHAR(20) UNIQUE NOT NULL,
 student_name VARCHAR(100) NOT NULL,
department VARCHAR(60) NOT NULL,
semester TINYINT NOT NULL
      CHECK (semester BETWEEN 1 AND 8)
);
```

This creates the students table with a primary key, unique roll number, and required student details.

7.2 Insert and Update Records

The INSERT command adds new student records

```
INSERT INTO students
(roll_no, student_name, department, semester)
VALUES
('BCA001', 'Ananya', 'BCA', 4);
```

The UPDATE command modifies existing records.

```
UPDATE students
SET semester = 5
WHERE roll_no = 'BCA001';
```

The SELECT command retrieves student information.

```
SELECT *
FROM students;
```

The DELETE command removes a record when permitted by the database relationships.

```
DELETE FROM students
WHERE roll_no = 'BCA001';
```

a student with related examination records cannot be deleted until the foreign-key relationships are handled appropriately.

7.3 Marks and Result Calculation

The system stores marks, calculates total marks and percentages, assigns grades, and determines pass or fail status. Absent students are recorded as absen

7.4 Student and Subject Reports

Student-wise reportsdisplayindividual examination results. Subject-wise reports show student performance in each subject. Aggregate functions such as SUM(), AVG(), MAX(), and MIN() help analyze marks.

7.5 Views and Stored Proced

Views display combined examination results in an organized format. Stored procedures help perform predefined database operations efficiently.

7.6 Transactions and Indexing

Transactions help maintain data consistency during mark entry and updates. Indexes improve the speed of searching and retrieving examination records.

7.7 Testing and Validation

The system is tested using valid and invalid data. Duplicate entries, marks outside the permitted range, invalid dates, and missing records are checked to ensure accurate results.

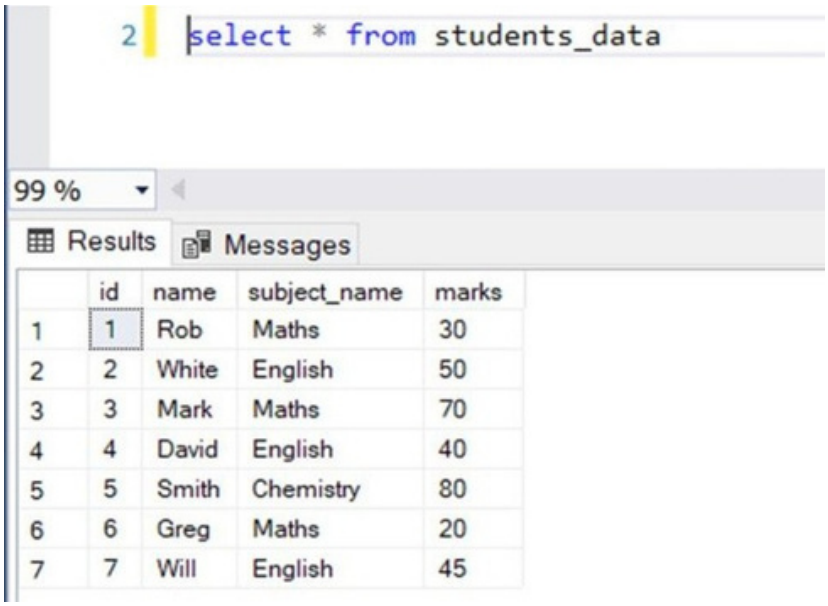
Test case	Expected behaviour
Obtained marks = 0	Accepted
Obtained marks = maximum marks	Accepted
Obtained marks below 0	Rejected
Obtained marks above maximum	Rejected
Duplicate mark entry	Rejected
Invalid student ID	Rejected by foreign key
Invalid examination schedule	Rejected by foreign key
Student absent	Stored with ABSENT status and NULL marks
Examination end date before start date	Rejected

8. Output Screenshots

This section contains the output screenshots associated with the project.
The following are sample database output captures produced from executed local test queries. They demonstrate the expected results, but they are not screenshots of MySQL Workbench.

8.1 Marks Table

Shows student roll numbers, subject codes, marks obtained, and attendance status.



```
2  select * from students_data
```

99 %

Results | Messages

	id	name	subject_name	marks
1	1	Rob	Maths	30
2	2	White	English	50
3	3	Mark	Maths	70
4	4	David	English	40
5	5	Smith	Chemistry	80
6	6	Greg	Maths	20
7	7	Will	English	45

8.2 Student Result Report

Displays student examination grades and pass/fail results.

	Roll_No	Semester	Subject	Marks	Grade	Result
0	101	Sem1	Maths	80	B	Pass
1	101	Sem1	Physics	58	C	Pass
2	101	Sem1	Chemistry	65	C	Pass
3	101	Sem2	BEEE	66	C	Pass
4	101	Sem2	Civil	86	A	Pass
...	...	...	...	...	...	...
85	110	Sem2	Civil	77	B	Pass
86	110	Sem2	C language	71	B	Pass
87	110	Sem3	DSA	55	C	Pass
88	110	Sem3	Python	79	B	Pass
89	110	Sem3	Deld	69	C	Pass

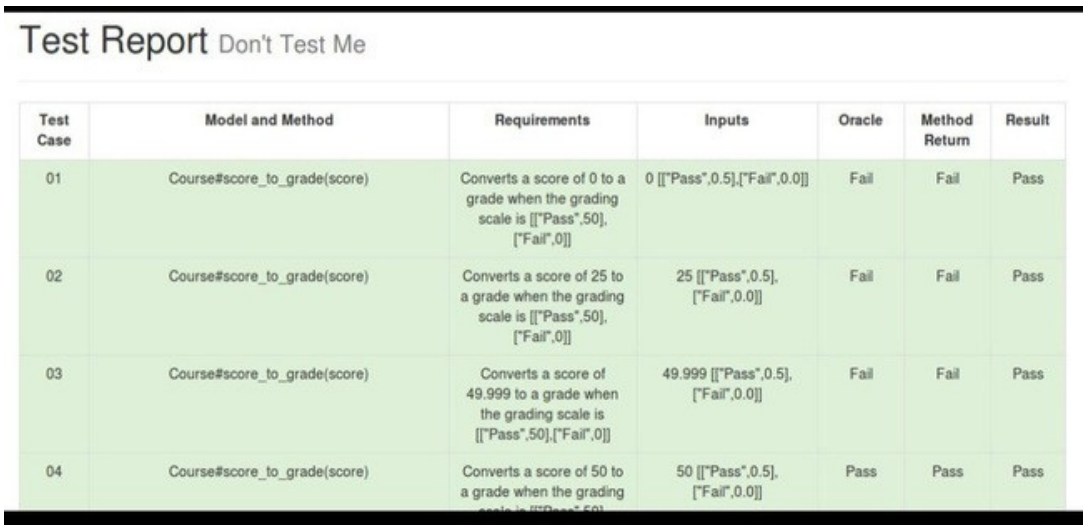
8.3 Subject-Wise Performance Report

Displays average, highest, and lowest marks and pass percentages.

Subject	Average Score	Max Score	Min Score	Standard Deviation	Pass Rate
Math	85.6	98	70	7.2	92%
Science	78.2	94	60	8.5	85%
History	92.1	100	80	4.9	98%
English	87.4	96	75	6.3	91%
Geography	81.9	90	70	5.8	88%

8.4 Boundary and Integrity Tests

Shows test outcomes for zero marks, duplicate records, invalid marks, and invalid student references.



Test Report Don't Test Me

Test Case	Model and Method	Requirements	Inputs	Oracle	Method Return	Result
01	Course#score_to_grade(score)	Converts a score of 0 to a grade when the grading scale is [["Pass",50], ["Fail",0]]	0 [["Pass",0.5],["Fail",0.0]]	Fail	Fail	Pass
02	Course#score_to_grade(score)	Converts a score of 25 to a grade when the grading scale is [["Pass",50], ["Fail",0]]	25 [["Pass",0.5], ["Fail",0.0]]	Fail	Fail	Pass
03	Course#score_to_grade(score)	Converts a score of 49.999 to a grade when the grading scale is [["Pass",50],["Fail",0]]	49.999 [["Pass",0.5], ["Fail",0.0]]	Fail	Fail	Pass
04	Course#score_to_grade(score)	Converts a score of 50 to a grade when the grading	50 [["Pass",0.5], ["Fail",0.0]]	Pass	Pass	Pass

For a submission that specifically requires MySQL Workbench screenshots, execute the supplied MySQL script in Workbench and capture the actual application windows and query results.

9. Challenges Faced

- Duplicate mark entries: Prevented using a unique constraint on student ID, subject ID, and examination ID.
- Invalid marks: Addressed using validation rules and MySQL triggers.
- Absent students: Managed using a separate attendance status and NULL marks.
- Incorrect result calculations: Addressed using SQL CASE expressions and aggregate functions.
- Relationships between tables: Maintained using primary keys and foreign keys.
- Performance of searches: Improved using indexes on frequently searched columns.

These solutions help maintain accurate and consistent examination records.

10. Future Enhancements

The project can be improved in the following :

- Add a web-based interface for faculty members and administrators.
- Introduce separate login accounts and role-based access control.
- Add an audit log to record changes made to student marks.
- Generate downloadable student mark sheets and report cards.
- Add charts and dashboards to display academic performance.
- Support automated database backups and recovery
- Store grading rules in a separate table so administrators can modify them easily.
- Add notifications when examination results are published.

These enhancements can make the system more useful for educational institut

11. Conclusion

The Student Examination Management System demonstrates the use of relational database concepts to manage student examination information efficiently.

The project includes related tables, primary keys, foreign keys, constraints, SQL queries, joins, aggregate functions, views, stored procedures, transactions, and indexes. It also provides logic for validating marks, handling absent students, calculating grades, and generating student-wise and subject-wise reports.

The system reduces duplicate entries, improves data accuracy, and simplifies the preparation of examination results. Through this project, practical knowledge of SQL and database administration can be applied to a real-world academic requirement.